# COMP3010 Set Exercises

## ***Task*** (Pretending to be malicious) – 10745379

Fake malicious behaviour chosen – *WGET/Download Chrome executable.*

## <u>Timeline:</u>

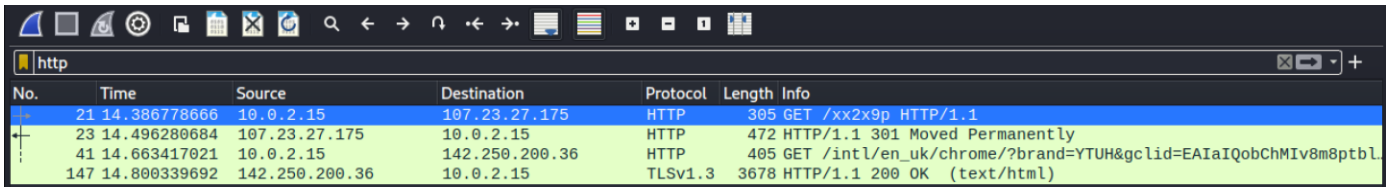
1. First set up the work environment by logging onto a virtual machine (I used Kali Linux).

2. Then install and configure tor and proxychains to work within your OS.

3. After, use the command '*export SSLKEYLOGFILE=/home/kali/Desktop/keylog*'. SSLDump was used in order to create the SSL Key Log File, allowing parts of the PCAP to be decrypted so tracing the malicious behaviour would be easier. The SSLKYLOGFILE environment variable that helps to read the PCAP file easier, wouldn't be used in real malicious behaviour.

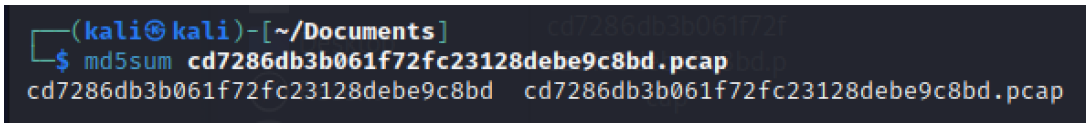
4. Finally, run Wireshark and I entered the command '*sudo ssldump -k $SSLKEYLOGFILE -i lo host rb.gy and port 443 & wget --secure-protocol=auto --https-only –user-agent="Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:95.0) Gecko/20100101 Firefox/95.0" --header="X-Custom-Header: Chrome Installer" https://rb.gy/ybabw4*'. (Explained further in Obfuscation Methods section).

5. Use the filter to see the HTTP requests.
   *ensure to add SSL key log file into TLS in Wireshark preferences



| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 21 | 14.386778666 | 10.0.2.15 | 107.23.27.175 | HTTP | 305 | GET /xx2x9p HTTP/1.1 |
| 23 | 14.496280684 | 107.23.27.175 | 10.0.2.15 | HTTP | 472 | HTTP/1.1 301 Moved Permanently |
| 41 | 14.663417021 | 10.0.2.15 | 142.250.200.36 | HTTP | 405 | GET /intl/en_uk/chrome/?brand=YTUH&gclid=EAIaIQobChMIv8m8ptbl |
| 147 | 14.800339692 | 142.250.200.36 | 10.0.2.15 | TLSv1.3 | 3678 | HTTP/1.1 200 OK (text/html) |

6. After saving the PCAP, convert it into MD5 hash using the command 'd5sum <file directory>\<PCAP name>.pcap'.



```
┌──(kali㉿kali)-[~/Documents]
└─$ md5sum cd7286db3b061f72fc23128debe9c8bd.pcap
cd7286db3b061f72fc23128debe9c8bd  cd7286db3b061f72fc23128debe9c8bd.pcap
```

In the screenshot taken below, in the subtrees you can see that the GET request data was received from the network under the custom user agent and custom header used (comp3010).

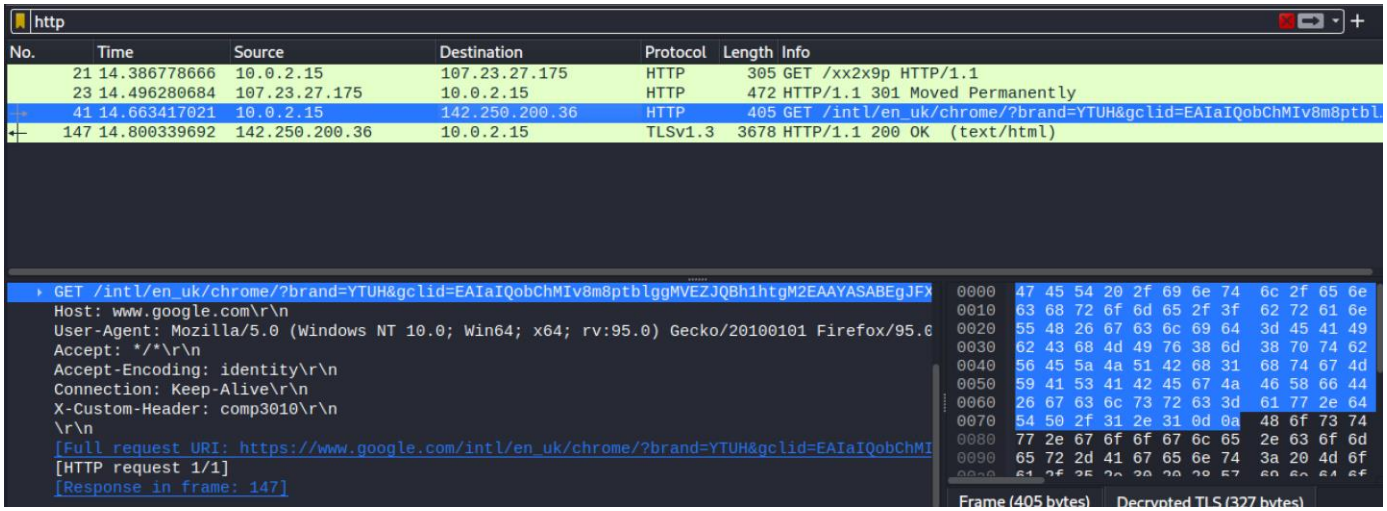
| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 21 | 14.386778666 | 10.0.2.15 | 107.23.27.175 | HTTP | 305 | GET /xx2x9p HTTP/1.1 |
| 23 | 14.496280684 | 107.23.27.175 | 10.0.2.15 | HTTP | 472 | HTTP/1.1 301 Moved Pe |
| 41 | 14.663417021 | 10.0.2.15 | 142.250.200.36 | HTTP | 405 | GET /intl/en_uk/chrom |
| 147 | 14.800339692 | 142.250.200.36 | 10.0.2.15 | TLSv1.3 | 3678 | HTTP/1.1 200 OK (tex |

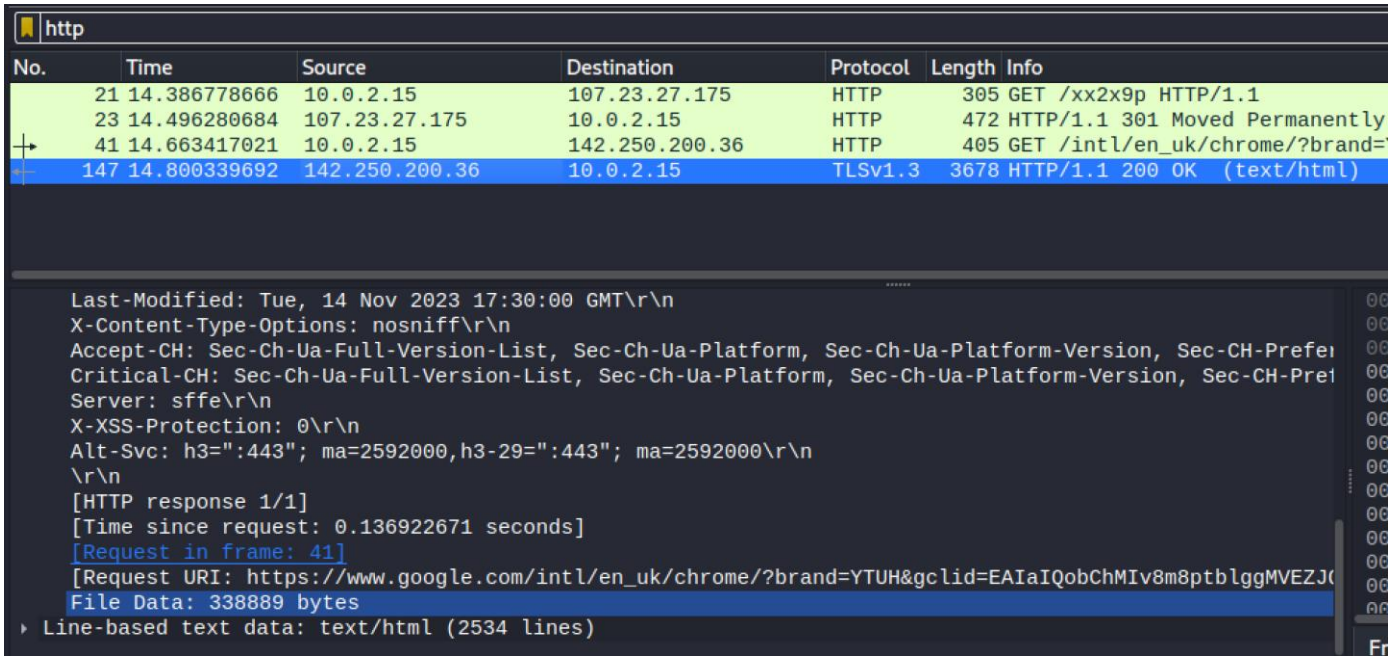
```
GET /xx2x9p HTTP/1.1\r\n
Host: rb.gy\r\n
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:95.0) Gecko/20100101 Firefox/95.0
Accept: */*\r\n
Accept-Encoding: identity\r\n
Connection: Keep-Alive\r\n
X-Custom-Header: comp3010\r\n
\r\n
[Full request URI: https://rb.gy/xx2x9p]
[HTTP request 1/1]
[Response in frame: 23]
```

Here (below) you can see that the user has been redirected the actual URL for the Chrome executable.

| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 21 | 14.386778666 | 10.0.2.15 | 107.23.27.175 | HTTP | 305 | GET /xx2x9p HTTP/1.1 |
| 23 | 14.496280684 | 107.23.27.175 | 10.0.2.15 | HTTP | 472 | HTTP/1.1 301 Moved Permanently |
| 41 | 14.663417021 | 10.0.2.15 | 142.250.200.36 | HTTP | 405 | GET /intl/en_uk/chrome/?brand= |
| 147 | 14.800339692 | 142.250.200.36 | 10.0.2.15 | TLSv1.3 | 3678 | HTTP/1.1 200 OK (text/html) |

```
HTTP/1.1 301 Moved Permanently\r\n
Date: Tue, 28 Nov 2023 02:52:05 GMT\r\n
Content-Length: 0\r\n
Connection: keep-alive\r\n
Cache-Control: no-cache, no-store\r\n
Expires: -1\r\n
Location: https://www.google.com/intl/en_uk/chrome/?brand=YTUH&gclid=EAIaIQobChMIv8m8ptblg
Engine: Rebrandly.redirect, version 2.1\r\n
Strict-Transport-Security: max-age=15552000\r\n
\r\n
[HTTP response 1/1]
[Time since request: 0.109502018 seconds]
```



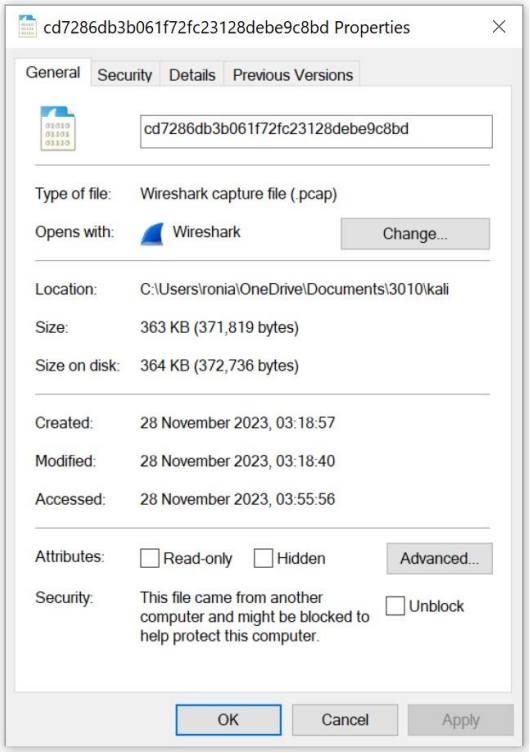

## Malicious Actions:

My ‘malicious’ action was to use WGET to download a Chrome executable. The attempt was harmless for obvious reasons and is only being used to emphasise how it could be used in a potentially malicious scenario, e.g., a user could unknowingly download a harmful executable such as a file with malware on it. On the other hand, this could also be used in ethical ways, such as in ethical hacking, to predict how attacks can be made or to educate and prepare others on what certain attacks will look like and how to avoid them.

## File Properties:

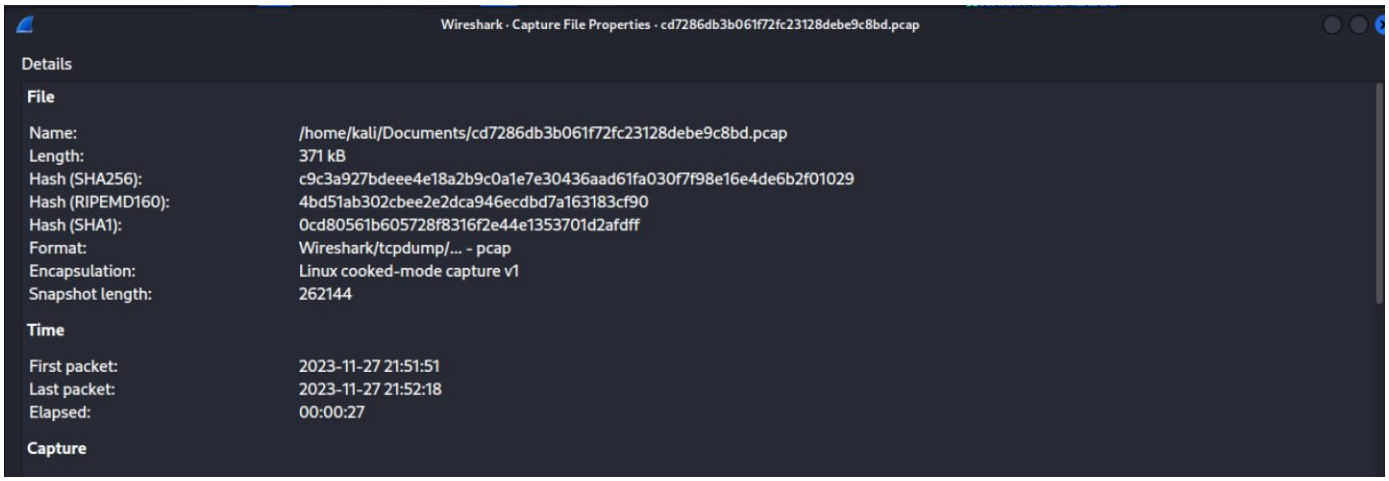
Name: cd7286db3b061f72fc23128debe9c8bd

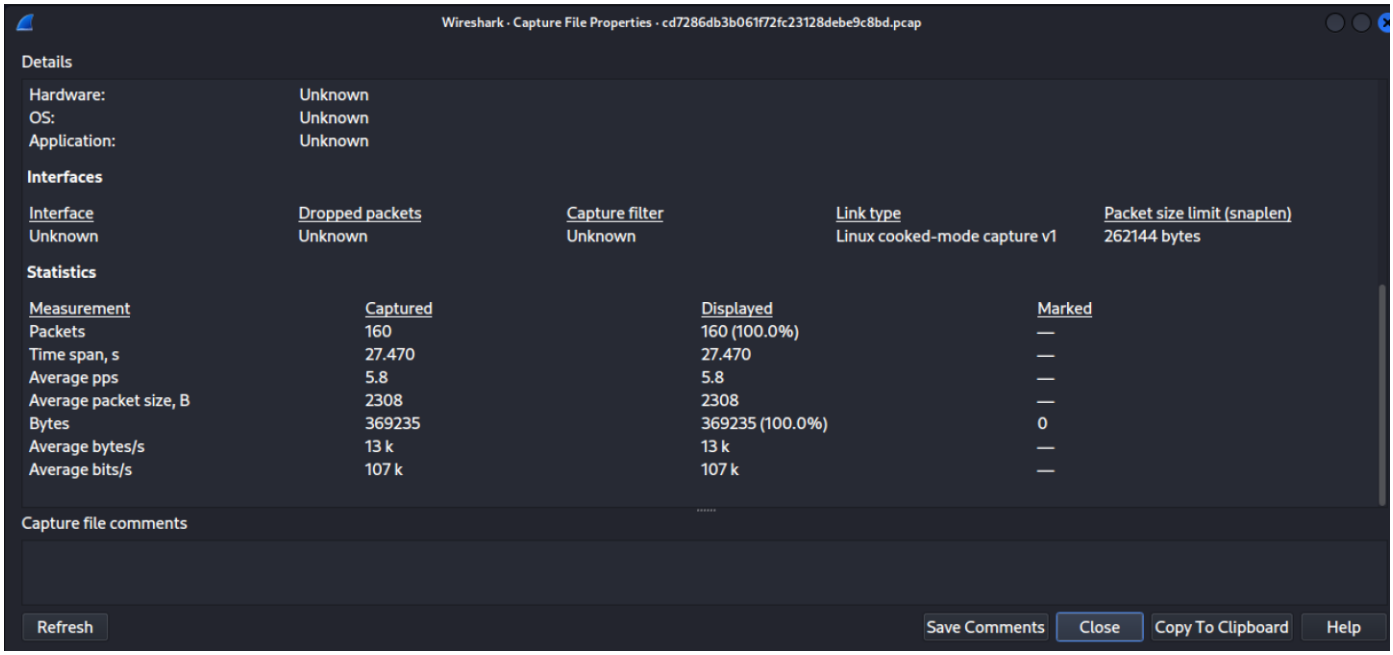
File size: 363 KB




## PCAP File Properties:

*Please see the Wireshark Capture File Properties (PCAP Properties) below.






## Timestamps:

First packet: 2023-11-27 21:51:51

Last packet:2023-11-27 21:52:18

Elapsed time: 00:00:27

## Other details:

Format: Wireshark/tcpdump/... – pcap

Encapsulation: Linux cooked-mode capture v1

Snapshot length: 262144

OS: Kali Linux 2023.3

Packets captured: 160

Time Span: 27.470

Total bytes captured: 369235

## SSL keys used:

These were used in the Key Log File to decode some traffic within the PCAP file.

CLIENT_RANDOM
62612927c8073730b725e9fcfa5666b6f90378b7b1eff06769428722e666b7a5
5e88d73009cc1b136deec13cd78a39ab7f4d5317221749fa86fa86891d9f11bce5f4dd0b24eb
8eadea4991fdcf5bf332

CLIENT_HANDSHAKE_TRAFFIC_SECRET
9af220c9fb03c875fd5c321aa65fac10cf7f9e8539dcc214e79ed780c94c08ed
9d00cfb6bb962ee92a24efca75efeafd10a12c474e1d7ceda59e17356e85f9ea4e1d524aaa9e1
6c98598d9ce716d1915
SERVER_HANDSHAKE_TRAFFIC_SECRET
9af220c9fb03c875fd5c321aa65fac10cf7f9e8539dcc214e79ed780c94c08ed
4080cc0fd7f712d801e05eb9b907a0764b828e025d8bf3fcd737aeb84fd020538c1a48d7cb7e
d2d94df82819eec246db

EXPORTER_SECRET
9af220c9fb03c875fd5c321aa65fac10cf7f9e8539dcc214e79ed780c94c08ed
dc899ea31ab87487542365853ba360a3b94e9761bd201f13927693a05d6f8f71649b44efb76
269127a67d6cdd8c65ef8

CLIENT_TRAFFIC_SECRET_0
9af220c9fb03c875fd5c321aa65fac10cf7f9e8539dcc214e79ed780c94c08ed
7443061d2e221aa84c8ef1d0887a4c654b91e0a5e5b0b7cb82c0e4c2c8189bdf3e2ed4a50bee
2d9ab3a7a70827e5b193

SERVER_TRAFFIC_SECRET_0
9af220c9fb03c875fd5c321aa65fac10cf7f9e8539dcc214e79ed780c94c08ed
1cec365350d8ee0bb878fe1a0bb524830fab5f9ddf460c29bd8e960c07e2f46fea922c8cb514a
9e8732580826b269b54

## Obfuscation Methods Used:

*'sudo ssldump -k $SSLKEYLOGFILE -i lo host rb.gy and port 443 & wget --secure-protocol=auto --https-only --useragent="Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:95.0) Gecko/20100101 Firefox/95.0" --header="X-Custom-Header: Chrome Installer" https://rb.gy/ybabw4'*

The command above was used for my 'malicious' behaviour, which can be seen as obfuscating in multiple ways.

1. *'wget --secure-protocol=auto --https-only'.* Ensures that the data will be encrypted and will ONLY travel through HTTPS(secure), helping to obfuscate data in packets.

2. *'—user-agent="Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:95.0) Gecko/20100101 Firefox/95.0" --header="X-Custom-Header: Chrome Installer'.* Using a user agent, as well as a custom header for good measure, will allow the request to seem more acceptable and harmless, making it harder to detect the potential malicious intent.

3. '*https://rb.gy/ybabw4'.* By using the shortened URL, the actual URL is hidden from the target.